CAF — Content Job Lifecycle

Aggregated export — 2026-06-16. Source markdown in repo is unchanged.

Bundle id: `10-caf-job-lifecycle`

Included: docs/JOB_LIFECYCLE.md

CAF — Content job lifecycle

Purpose: Complete guide to how a **content job** moves from planning through generation, QC, rendering, human review, rework, and publishing. For run-level lifecycle, see **LIFECYCLE.md**.

Key entity: `caf_core.content_jobs` — unique on `(project_id, task_id)`.
Source of truth for job JSON: `generation_payload` (+ columns `status`, `qc_status`, `render_state`).

Table of contents

---

1. Where a job comes from

A **content job** is created when a **run** is started and the decision engine selects a planned row.

```
Signal pack  →  Materialize planned rows on run  →  startRun  →
```

```
decideGenerationPlan  →  upsertContentJob
```

Step	What happens
Signal pack	Research bundle with ideas (`jobs_json` / legacy formats)
Materialize	`POST /v1/runs/:project_slug/:run_id/jobs` writes `runs.planned_jobs_json`
Start run	`POST .../start` scores candidates, applies caps, creates jobs
Job row	Inserted with `status = PLANNED`, `task_id` assigned, `generation_payload` seeded

`task_id` pattern:
`{run_id}__{platform}__{flow_type}__row{NNNN}__{variation}`
Example: `SNS_2026W09__Instagram__FLOW_CAROUSEL__row0002__v1`

Each job is **one idea × one flow type × one variation**.

---

2. The happy path (overview)

```
stateDiagram-v2
  [*] --> PLANNED: upsertContentJob
  PLANNED --> GENERATING: pipeline picks up
  GENERATING --> GENERATED: LLM output saved
  GENERATED --> RENDERING: QC pass + render lane
  RENDERING --> IN_REVIEW: assets ready
  IN_REVIEW --> APPROVED: human approves
  APPROVED --> [*]
```

Split-process mode (common in production):

1. **`POST .../process`** — LLM + QC + diagnostics → job stops at **`GENERATED`** ("package ready").
2. **`POST .../render`** — render lane → **`IN_REVIEW`**.

This separates expensive generation from expensive rendering.

---

3. Status reference

Primary job statuses

Status	Meaning
`PLANNED`	Job exists; waiting for pipeline
`GENERATING`	LLM generation in progress
`GENERATED`	Draft package ready (`generated_output` in payload); may await render
`RENDERING`	Media production in progress (carousel, video, mimic)
`IN_REVIEW`	Waiting for human editorial decision
`APPROVED`	Human approved — eligible for publish
`REJECTED`	Human or QC discarded the job
`NEEDS_EDIT`	Human or QC requested rework
`BLOCKED`	QC blocked (risk / checklist failure)
`QC_FAILED`	QC failed without a softer route
`FAILED`	Pipeline error (generation, render, timeout)

> ***`APPROVED`*** *is set only via* ***human review*** *— QC does not auto-approve by default (* *`CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC`* *).*

Editorial decisions (separate table)

Stored on **`caf_core.editorial_reviews.decision`**:

Decision	Sets job status
`APPROVED`	`APPROVED`
`REJECTED`	`REJECTED`
`NEEDS_EDIT`	`NEEDS_EDIT`

---

4. Stage-by-stage detail

4.1 PLANNED → GENERATING

Module: `src/services/job-pipeline.ts` → `advanceToGenerating`

- Pipeline picks up jobs in `PLANNED` (or retries `GENERATING`).
- Offline flow types exit early (`offline-flow-types.ts`).
- Transition logged to **`job_state_transitions`**.

4.2 GENERATING → GENERATED

Module: `src/services/llm-generator.ts` → `generateForJob`

Action	Detail
Prompt resolution	Flow engine templates per `flow_type` + project
Creation pack	Signal context, brand, strategy; mimic filters to single idea
Learning injection	`getLearningContextForGeneration` appends guidance
LLM call	OpenAI (or placeholder mode)
Schema validation	`CAF_OUTPUT_SCHEMA_VALIDATION_MODE`
Persist	`job_drafts` row + merge `generated_output` into `generation_payload`
Status	**`GENERATED`**

Mimic prep (when enabled): reference resolution may run during GENERATING before copy (`mimic-draft-prep.ts`).

4.3 GENERATED → QC

Module: `src/services/qc-runtime.ts` → `runQcForJob`

Input	Output
`generation_payload.generated_output`	`qc_result` slice (via **`mergeGenerationPayloadQc`**)
Flow checklist + risk policies + brand bans	`qc_status`, `recommended_route` on job row

See **QUALITY_CHECKS.md**.

4.4 Post-QC diagnostic

Module: diagnostic audit (machine evaluation snapshot on job).

Runs after QC on the standard path before render.

4.5 GENERATED → RENDERING → IN_REVIEW

Module: `job-pipeline.ts` render branches

Flow family	Render path
Carousel	HTTP → `RENDERER_BASE_URL` → PNG slides → `assets`
HeyGen video	Submit + poll → MP4 asset

Scene assembly	Sora clips + video-assembly stitch/mux
Mimic image/carousel	Image provider + HBS/DocAI overlays

On success: `finalJobStatusAfterRender` → `IN_REVIEW` (always human gate).

Publish URL helpers may merge into `generation_payload` (`publish_media_urls_json`, `publish_video_url`).

4.6 IN_REVIEW → terminal editorial status

Module: `src/routes/v1.ts` + Review app

Human decides → `editorial_reviews` row + `content_jobs.status` update.

---

5. QC routing branches

After `runQcForJob`, `routeJobAfterQc` (`validation-router.ts`) may short-circuit:

```
flowchart TD
  QC[runQcForJob]
  QC -->|recommended_route = DISCARD| REJ[REJECTED]
  QC -->|recommended_route = REWORK_REQUIRED| NE[NEEDS_EDIT]
  QC -->|qc_passed = false, BLOCKED| BL[BLOCKED]
  QC -->|pass| REN[Continue to render]
  REN --> IR[IN_REVIEW]
```

recommended_route	Typical next status	Render?
DISCARD	REJECTED	No
REWORK_REQUIRED	NEEDS_EDIT	No
BLOCKED	BLOCKED	No
HUMAN_REVIEW	Continue → render → IN_REVIEW	Yes (if QC passed)
AUTO_PUBLISH	Still → IN_REVIEW when human gate on	Yes

---

6. Rendering paths

Carousel (`FLOW_CAROUSEL`)

1. Build render pack from `generated_output` + templates.
2. POST slides to carousel renderer.
3. Upload PNGs to Supabase → `assets` rows.
4. Job → `IN_REVIEW` .

Video (HeyGen / scene)

1. Submit render job to provider.
2. Job may stay `RENDERING` during long polls.
3. `hasActiveProviderSession` prevents double-submit on retry.
4. `RenderNotReadyError` → stay `RENDERING` , safe to retry later.
5. On completion → asset + `IN_REVIEW` .

Mimic (`FLOW_TOP_PERFORMER_MIMIC_*`)

Same status enum. Extra prep on `generation_payload.mimic_v1` before/after copy. Render produces `MIMIC_BACKGROUND` , `MIMIC_VISUAL_PLATE` , or carousel slide assets.

Detail: **MIMIC_FLOWS_COMPLETE_GUIDE.md**.

---

7. Human review

Surfaces: Review app (`apps/review`) or `/v1` review APIs.

Capability	Description
Queue	List jobs in `IN_REVIEW` (per project or all projects)
Detail	Full `generation_payload`, assets, QC snapshot
Decision	Approve / reject / needs edit
Overrides	Script, slides, HeyGen ids, skip regen flags, mimic layer positions

Post-approval (optional): LLM approval review → scores + `upstream_recommendations` → learning observations.

---

8. Rework loop

When status is `NEEDS_EDIT` :

```
flowchart LR
  NE[NEEDS_EDIT]
  RO[rework-orchestrator]
  GEN[GENERATING / GENERATED]
  REN[RENDERING]
  IR[IN_REVIEW]

  NE --> RO --> GEN --> REN --> IR
```

Module: `src/services/rework-orchestrator.ts` → `executeRework`

Rework type	What re-runs
Copy rewrite	LLM generation
Slide edits	Partial carousel re-render
Video script	HeyGen re-submit (respects `skip_video_regeneration`)
Mimic text overlay	Text reprint path (may stay `IN_REVIEW` during reprint)

Reviewer notes and `editorial_overrides_json` feed back into the user prompt on rework.

---

9. After approval: publish

Job status: `APPROVED`

Publishing is a separate lifecycle on **`publication_placements`**:

`draft` → `scheduled` → `publishing` → `published` | `failed` | `cancelled`

Linked by **`(project_id, task_id)`** — not a FK.

Executor modes: `none` (external worker), `dry_run`, `meta` (in-Core Graph API).

Detail: layers/publishing.md.

---

10. Failure & retry semantics

Situation	Behavior
LLM / render exception	`FAILED` + `job_state_transitions`

HeyGen still polling	Stay `RENDERING`; use `hasActiveProviderSession` before re-submit
Run process retry	`GENERATED` jobs picked up for render; safe `RENDERING` retries per flow rules
QC block	`BLOCKED` — manual intervention
Mimic text reprint stuck	May mark `FAILED` explicitly vs silent `RENDERING`

Run-level processing scans jobs in `PLANNED`, `GENERATING`, `GENERATED`, `RENDERING` and advances what is eligible.

---

11. Audit trail

Store	Contents
`job_state_transitions`	Every status change (`from_state` → `to_state`)
`job_drafts`	Each LLM attempt (`attempt_no`, `revision_round`)
`editorial_reviews`	Human decisions + overrides
`diagnostic_audits`	Machine quality snapshots
`assets`	Render outputs with versions
`api_call_audit`	External API calls (optional cost)

Correlation: trace by `task_id` + `project_id` across all tables.

---

12. APIs & operators by stage

Stage	API / surface
Create jobs	`POST /v1/runs/:project_slug/:run_id/start`
Process (generate+QC)	`POST /v1/runs/.../process`, `POST /v1/pipeline/...`, `npm run process-run`
Render	`POST /v1/runs/.../render`
Single job	`POST /v1/jobs/:project_slug/:task_id/process`
Review queue	`/v1/review-queue...`, Review app workbench
Editorial decision	`POST /v1/reviews` (see `v1.ts`)
Rework	Pipeline rework routes → `rework-orchestrator`
Publish	`/v1/publications/:project_slug/...`

---

13. Related docs

Doc	Topic
LIFECYCLE.md	Run lifecycle + publication + learning rule states
layers/job-pipeline.md	Execution layer modules
layers/review-rework.md	Review API + rework
DOMAIN_MODEL.md	IDs and entity relationships
QUALITY_CHECKS.md	QC runtime detail
MIMIC_FLOWS_COMPLETE_GUIDE.md	Mimic job path

Job lifecycle — from `PLANNED` to `APPROVED`, with QC gates, human review, and rework loops at every step.

Included: docs/LIFECYCLE.md

CAF Core — Lifecycles

State machines for **runs**, **content jobs**, and how they connect to **review** and **publishing**. Status strings are **text** in Postgres; clients must not invent new values without migration and pipeline updates.

Run lifecycle

Runs live in `caf_core.runs`. The orchestrator in `src/services/run-orchestrator.ts` drives planning when you call `POST /v1/runs/:project_slug/:run_id/start`.

Phase	Typical `runs.status`	What happens
Created	`CREATED`	Run row exists; `signal_pack_id` should be set before start (via create or `PATCH`).
Planning	`PLANNING`	Signal pack loaded; candidates built; `decideGenerationPlan` runs; `content_jobs` inserted `PLANNED`.
Planned / generating	`PLANNED` then `GENERATING`	`startRun` sets `PLANNED` (total_jobs) then `GENERATING` after jobs exist.
Execution	`GENERATING` → `RENDERING` → `REVIEWING`	Updated by `job-pipeline` / run processing as jobs advance (see run repository).

Terminal	`COMPLETED`, `FAILED`, `CANCELLED`	Failure on planning errors; **`COMPLETED`** when work finishes with zero or more jobs.

Allowed values are constrained in SQL (see **`migrations/002_project_config_and_runs.sql`**): `CREATED`, `PLANNING`, `PLANNED`, `GENERATING`, `RENDERING`, `REVIEWING`, `COMPLETED`, `FAILED`, `CANCELLED`.

Important: **`startRun`** requires **`signal_pack_id`** on the run. **`CREATED`** runs with no pack will fail start.

Important (current wiring): **`startRun`** also expects planner rows to already exist on the run as **`runs.candidates_json`**.

- Create/materialize them via **`POST /v1/runs/:project_slug/:run_id/candidates`** while the run is still `CREATED`.
- `start` will error if `candidates_json` is missing/unusable.

Content job lifecycle

Jobs are **`caf_core.content_jobs`**, unique on **`(project_id, task_id)`**.

Full guide: [JOB_LIFECYCLE.md] — stage-by-stage from `PLANNED` through review, rework, and publish.

Typical pipeline path (**`src/services/job-pipeline.ts`**):

```
PLANNED → GENERATING → GENERATED → (QC) → RENDERING → IN_REVIEW →
APPROVED | REJECTED | NEEDS_EDIT
```

Notes:

- **`GENERATING`** — job picked up; LLM may run.
- **`GENERATED`** — LLM output present in **`generation_payload`**.
- After **`runQcForJob`**, status may become **`BLOCKED`**, **`QC_FAILED`**, **`REJECTED`**, **`NEEDS_EDIT`**, or advance toward render/review depending on **`recommended_route`** and flow type.
- **`IN_REVIEW`** — waiting for human decision (default path after QC + render when using **`CAF_REQUIRE_HUMAN_REVIEW_AFTER_QC`**).

- `APPROVED` / `REJECTED` / `NEEDS_EDIT` — set from review APIs (`src/routes/v1.ts`). `NEEDS_EDIT` often triggers **rework** (`rework-orchestrator.ts`).

QC can short-circuit: `routeJobAfterQc` maps `DISCARD` → `REJECTED`, `REWORK_REQUIRED` → `NEEDS_EDIT`.

Video jobs may stay `RENDERING` for a long time while HeyGen/Sora poll; `RenderNotReadyError` keeps status `RENDERING` for retry.

Top-performer mimic jobs (`FLOW_TOP_PERFORMER_MIMIC_*`) follow the same status enum. Mimic prep may run during **GENERATING** (reference resolve before copy); render produces assets then lands in `IN_REVIEW` like other carousel/image jobs. See MIMIC_IMAGE_FLOWS.md.

Editorial decision (human)

Stored in `caf_core.editorial_reviews`; `decision` is one of:

- `APPROVED`
- `NEEDS_EDIT`
- `REJECTED`

Applied via review endpoints that update `content_jobs.status` to match.

Publication placement lifecycle

`caf_core.publication_placements`: `status` values include `draft`, `scheduled`, `publishing`, `published`, `failed`, `cancelled` (see `src/routes/publications.ts`). Linked to jobs by `(project_id, task_id)`, not a FK to `content_jobs`.

Learning rules (lifecycle)

`caf_core.learning_rules`: `status` includes `pending`, `active`, `superseded`, `rejected`, `expired`. Activation `applyLearningRule` sets `active` and `applied_at`. Only **active** rules with valid `valid_from` / `valid_to` / `expires_at` participate in planning or generation (see `GENERATION_GUIDANCE.md` and `src/repositories/core.ts`).

Where to trace in code

Concern	File
Start run, create jobs	`src/services/run-orchestrator.ts`
Process jobs	`src/services/job-pipeline.ts`
QC routing after pass/fail	`src/services/validation-router.ts`, `qc-runtime.ts`
Human review	`src/routes/v1.ts`
State transition log	**`caf_core.job_state_transitions`** via **`src/repositories/transitions.ts`**

Related docs

- ARCHITECTURE.md — abbreviated lifecycle
- layers/job-pipeline.md — execution layer
- MIMIC_IMAGE_FLOWS.md — mimic job path
- .cursor/rules/caf-domain-model.mdc — ID conventions

Included: docs/layers/job-pipeline.md

Layer: Job pipeline (execution)

Purpose: Move a single `content_job` through **generation** → **QC** → **diagnostic** → **render** → **review-ready status**, and run **batch** processing for a whole run.

Main module

- `src/services/job-pipeline.ts` — `processContentJobById`, `processRunJobs`, `reprocessJobFromScratch`, carousel/video branching, **HeyGen/Sora** retry semantics.

Stages (conceptual)

1. **Offline flows** — early exit for types in `offline-flow-types.ts`.
2. **PLANNED → GENERATING** — `advanceToGenerating`.
3. **LLM** — `generateForJob` if no `generated_output` yet.
4. **QC** — `runQcForJob` persists results through `mergeGenerationPayloadQc` (`src/domain/generation-payload-qc.ts`, typed by `qcResultSchema`) — see **../QUALITY_CHECKS.md**.
5. **Post-QC routing** — `routeJobAfterQc` (`validation-router.ts`) for DISCARD / REWORK.
6. **Diagnostic** — `runDiagnosticAudit`.

7. **Render** — carousel slides via `RENDERER_BASE_URL`; **mimic carousel/image** via `mimic-image-provider.ts` + HBS/DocAI overlays when `MIMIC_IMAGE_ENABLED`; video via HeyGen / scene pipeline / video-assembly (flow-dependent).
8. **Terminal pre-review** — `IN_REVIEW`, `finalJobStatusAfterRender`.

Inputs

- `job.id` (UUID) or `task_id` + project; `AppConfig` for URLs and flags.

Outputs

- Updates `content_jobs` (status, `generation_payload`, `render_state`, `scene_bundle_state`, `asset_id`).
- `caf_core.assets` inserts.
- `job_state_transitions` rows.

State owned

- Job row is SSOT; pipeline **merges** JSON into `generation_payload`. The `qc_result` slice has a canonical writer (`mergeGenerationPayloadQc`); other subsystems update their slices directly via SQL — coordinate changes.

Failure modes

- `RenderNotReadyError` — poll timeouts; job may stay `RENDERING`.
- `markJobFailedPipeline` — `FAILED` with transition log.

Provider idempotency (HeyGen / Sora)

The canonical "don't double-submit" check is `hasActiveProviderSession(renderState)` in `src/domain/content-job-render-state.ts`. The pipeline uses it in `isVideoRenderingSafelyRetryable`. New render branches must call this helper before any provider submit.

Observability

For new pipeline-stage logs, prefer `logPipelineEvent(level, stage, message, { run_id, task_id, job_id, flow_type, data })`

(`src/services/pipeline-logger.ts`) over `console.*` . It emits a single JSON line per event with correlation fields, so one job is easy to trace across generate → QC → render → review in container log collectors.

Boundaries

- **Depends on:** `llm-generator` , `qc-runtime` , `diagnostic-runner` , `validation-router` , render helpers, **Supabase** upload, `mimic-draft-prep` , `mimic-carousel-render` , `mimic-image-job` (when mimic flows enabled).
- **See:** generation.md, rendering.md, ../MIMIC_IMAGE_FLOWS.md, ../LIFECYCLE.md.

Included: docs/layers/review-rework.md

Layer: Review & rework

Purpose: Record **human decisions** on jobs, update **status**, and drive **rework** (partial/full regeneration, caption-only video paths, HeyGen overrides).

Review (API)

- `src/routes/v1.ts` — review queue, job detail, `executeEditorialReviewDecision` pattern: inserts `caf_core.editorial_reviews` , updates `content_jobs.status` to `APPROVED` , `REJECTED` , or `NEEDS_EDIT` , `insertJobStateTransition` .
- Body may include **overrides**: title, hook, caption, hashtags, slides JSON, **spoken script**, HeyGen ids, `skip_video_regeneration` , `skip_image_regeneration` , `rewrite_copy` , etc.

Rework execution

- `src/services/rework-orchestrator.ts` — `executeRework` , prepares payload for another pipeline pass (linked from `pipeline.ts` routes).
- When reading the prior draft off `generation_payload.generated_output` , prefer `pickGeneratedOutput / pickGeneratedOutputOrEmpty` from `src/domain/generation-payload-output.ts` so rework code does not silently treat missing output as an empty object.

Review app

- `apps/review` — Next.js; **proxies** Core (`caf-core-client.ts`). **Source of truth remains Postgres via Core**, not the Review DB.
- **Mimic carousel jobs** — `MimicCarouselEdits` , DocAI layer editor, mimic mode overrides on signal packs (`/api/pipeline/signal-packs/.../mimic-mode-override`). See **../MIMIC_IMAGE_FLOWS.md**.

Inputs

- `project_slug` , `task_id` , decision + optional overrides.

Outputs

- `editorial_reviews` row; `content_jobs` status and merged `generation_payload` / `overrides_json` as applicable.

State owned

- **Editorial** history on `editorial_reviews` ; job **current** status on `content_jobs` .

See also

- ../LIFECYCLE.md
- ../GENERATION_GUIDANCE.md (rework prompt injection)
- publishing.md